User Flows & Behavioral Specification

Version: 2025-08-23 Scope: Current codebase (web/ + server/) for Fitness Tracker PWA Audience: Product, Engineering, QA

1. High-Level Architecture

- Client: Vanilla JS single-page shell (`web/index.html`) loading `extras.js` (UX enhancements + diagnostics) then `app.js` (core logic).
- Backend: Express API (`server/server.js`) with JWT auth; resources under `/api/*` (auth, entries, summary, me).
- Persistence (client): localStorage (token + offline outbox queue) + in-memory fallback + (future option: IndexedDB).
- Offline strategy: Service Worker (`web/service-worker.js`) caches shell/assets (static cache) + network-first navigations + queue for POST during offline.
- AuthN: Register/Login -> receive JWT -> stored in localStorage key `ft.jwt`.
- AuthZ: JWT verified per request (`requireAuth` middleware) mapping user id to entries & summary.

```
[User] -> UI (index.html) -> app.js api() -> fetch /api/... -> Express -> DB
                                ^         \
                                |          -> queueOutbox() (if offline)
                Service Worker (cache shell/assets)
```

2. Primary User Flows

2.1 Registration

Trigger: User clicks Register button on auth card. Preconditions: User not authenticated (no valid `ft.jwt`). Steps:

1. Collect `email`, `password` inputs.
2. POST `/api/auth/register` { email, password }.
3. On 200: store `token` in localStorage (`ft.jwt`), call `bootApp()`.
4. On 400 (duplicate or validation): display alert with error. Postconditions: Auth container hidden, main app visible, stations grid built, today's entry loaded (if any new entry returns null, counts start at 0). Errors: Network failure -> alert. 401 unexpected -> fallback to login view. Security: Password sent over HTTPS (assumed). No password stored locally.

2.2 Login

Trigger: User clicks Login. Steps identical to Registration except endpoint `/api/auth/login` and 401 returns "Invalid credentials". Edge Cases: Rapid repeated invalid attempts—no explicit lockout (future enhancement).

2.3 Boot (Initial Authenticated Session)

Invoked: After successful auth OR page load with existing token. Steps:

1. Hide auth, show app containers.
2. Display user email (`/api/me` if not supplied by auth response).
3. Build exercise station cards.
4. Initialize counts (0) & compute progress.
5. Flush offline outbox (`flushOutbox`).
6. Load today's entry (`/api/entries/:date`); hydrate counts & weight.
7. Update progress bar.
8. Dispatch `ft:booted` event (legacy integration hook). Failure: Any critical error -> clear token, revert to auth view.

2.4 Daily Progress Update (Increment/Decrement Sets)

Trigger: User taps + or - on station. Steps:

1. Delegated click listener reads `data-name` & `data-delta`.
2. `updateCount()` adjusts bounded 0..MAX_SETS, updates UI + enable/disable buttons, toggles `.completed` class, recalculates progress. No server interaction until Save.

2.5 Save Progress

Trigger: User clicks Save Progress. Steps:

1. Gather counts + weight -> payload { date, weight, fitnessData }.
2. Debounce: If identical payload within 2s, ignore.
3. Call `api('/api/entries', POST …)`.
    - If offline: queued -> toast "Offline: queued to sync".
    - If online success: toast "Progress saved!".
4. On error: toast "Save failed: ". Outbox Logic: For `/api/entries`, existing queued same date replaced (idempotent per-day update model).

2.6 View History (Editable List)

Trigger: User clicks View History. Steps:

1. `showHistory()` GET `/api/entries`.
2. De-dup on date (Map keeps last row per `entry_date`).
3. Sort ascending (oldest -> newest during build; could be reversed in UI if desired).
4. Render editable rows (weight + per-exercise sets inputs + save/delete buttons).
5. Open modal + attach click handlers to dynamic save/delete buttons. Inline Edit Save: Rebuilds payload from inputs -> POST same endpoint (upsert) -> toast Updated! -> refetch. Delete: DELETE `/api/entries/:date` -> toast Deleted -> refetch. Close: Modal close button or click outside area. Edge: Legacy placeholder rows (non YYYY-MM-DD) filtered out.

2.7 View Weekly Summary

Trigger: User clicks Summary. Steps:

1. `showSummary()` GET `/api/summary`.

2. For each week row (Sunday anchor) render: weight average + total sets per exercise + aggregate total.
3. Empty state: single-row message.
4. Modal behaviors / close match History. Fallback Query: If complex SQL fails, simple summary used (weight averages only, empty totals `{}`) ensuring resilience.

2.8 Logout

Trigger: Logout button. Steps: Clear token (`ft.jwt`), `location.reload()` (re-enters unauth state, shows auth card).

2.9 Offline Queue & Sync

Scenario: User offline saving progress. Flow:

1. User clicks Save -> `api()` detects offline for non-GET -> queueOutbox({ path:'/api/entries', method:'POST', body }).
2. Entry replaced in queue if same date.
3. On `online` event or next boot, `flushOutbox()` processes deduped queue in chronological order.
4. Failures remain in queue (partial sync) for next attempt. UX: Toast indicates queued. No explicit queue length display.

2.10 Dev Bypass Mode

Activation: `?dev=1`, `#dev`, or `localStorage.ft.devBypass=1` before load. Flow:

1. `DEV_BYPASS` true -> mock `api()` returns stub responses.
2. `bootApp('dev@example.com')` called directly, skipping auth.
3. Diagnostics banner displays bypass notice. Security: Not server-side; production server unaffected (safe for styling preview). Ensure flag removed before user acceptance testing.

2.11 Face ID Demo (Simulated Quick Login)

Trigger: Click Face ID icon. Flow: Adds CSS scanning state -> after timeout dispatches `ft:faceAttempt` -> attempts login using inputs or demo credentials -> boots app if success. Failure: Silent console warning.

2.12 Service Worker Lifecycle

Initial Load:

- `extras.js` registers `./service-worker.js` (scope `/web/`). Install: Caches static asset list (versioned `ft-static-v9`). Activate: Deletes old caches; posts `SW_ACTIVATED` message. Fetch Strategy:
- API: network only (with offline 503 JSON fallback).
- Navigations: network-first (ensures fresh HTML under CSP changes) with cached fallback.
- Assets: cache-first (risk: JS staleness until hard refresh). Update Flow:
- Deploy new version -> user receives old asset until SW update + refresh.
- Manual version bump required in cache name or query param for immediate propagation.

---

3. Error & Edge Case Handling

Area	Condition	Behavior	Improvement Opportunity
Auth Check	Stored token invalid	Clear token, show auth UI	Add user-facing message (expired session)
Offline Save	Navigator offline	Queue entry, toast	Display offline badge & queue size
Duplicate Save	Same payload <2s	Ignored	Provide subtle hint (already saved)
History Placeholder	Legacy non-date row	Filtered out	Add migration notice if any filtered
Summary Query Failure	Complex SQL fails	Fallback simple summary	Alert for admin monitoring
localStorage Unavailable	Private mode/quota	In-memory fallback w/o persistence	Show warning badge

4. Data Model (Client Perspective)

Entry Payload:

```
{
  date: 'YYYY-MM-DD',
  weight: number | null,
  fitnessData: {
    'Bike': number, 'Abs': number, ... (exercise name -> sets)
  }
}
```

Server Response (GET entries): array of

```
{ entry_date: 'YYYY-MM-DD', weight: number|null, fitness_data: { <exercise>: sets,
... } }
```

Summary Row:

```
{
  week_of_sunday: 'YYYY-MM-DD',
  weight_avg: string (numeric(6,2) cast),
  totals: { 'Bike': totalSets, ... }
}
```

5. Security Considerations

Current:

- JWT stored in localStorage (susceptible to XSS if introduced).
- Helmet default CSP: inline scripts were removed (reduces risk).
- No refresh tokens; single token lifespan (assumed long-lived). Recommendations:
- Explicit CSP policy locking `script-src 'self'` + `connect-src 'self'`.
- Consider `Authorization` header rotation / short-lived JWT with silent refresh.
- Input validation / rate limiting on auth endpoints.

---

6. Performance & UX Notes

- Stations built once; minimal DOM churn on increment.
- History/ Summary modals rebuilt on each open (simple, acceptable scale).
- Potential staleness: JS/CSS served cache-first — consider stale-while-revalidate.
- Large inline style block duplicates external CSS (candidate for consolidation / critical CSS extraction).

---

7. Accessibility / Progressive Enhancement

Current:

- Buttons and inputs use semantic elements. Gaps:
- No focus trapping or aria-modal attributes in modals.
- Insufficient role/aria-labels for progress bar and dynamic counters. Recommendations:
- Add `role="progressbar"` with `aria-valuenow`, etc.
- Add `role="dialog"` and `aria-modal="true"` to modal containers; handle Escape key to close.

---

8. Known Gaps & Backlog Candidates

1. Slugify exercise IDs to avoid space-containing DOM IDs.
2. UTC date normalization (reduce timezone ambiguity).
3. IndexedDB mirror for full offline history & summary.
4. User feedback on token expiration.
5. Service worker asset strategy modernization (hashed filenames).
6. Visible offline & sync status UI.
7. Accessibility enhancements (dialogs, focus management, ARIA roles).
8. Security hardening (contentSecurityPolicy explicit, rate limits, password policy feedback).

---

9. QA Test Matrix (Condensed)

Scenario	Steps	Expected
Register new user	Fill form -> Register	App boots; token stored; counts zero
Login existing	Fill credentials -> Login	App boots; today loads prior sets

Scenario	Steps	Expected
Save offline	Disable network -> Increment -> Save	Toast queued; on reconnect flush + persisted
Duplicate save debounce	Save twice within 2s	Second ignored; no duplicate network call
Edit history entry	Open History -> Modify sets -> Save icon	Row updates; toast Updated!
Delete entry	History -> Delete icon	Row removed; toast Deleted
Summary empty	New account -> Summary	Message: No workout data available yet
Dev bypass	Append ?dev=1	App boots w/o auth; email dev@example.com
Face ID demo	Click icon with valid demo creds	Automatic login & boot
Token invalidation	Manually corrupt token -> reload	Returns to auth screen
SW update	Increment version -> reload	New cached version used (verify Network)

10. Change Control & Versioning

- Manual asset versioning via query params `?v=diagX` and SW cache name `ft-static-v#`.
- Recommendation: migrate to automated build step generating hashed bundles + manifest.

---

11. Glossary

Term	Definition
Entry	A single day's workout + weight data
Outbox	Client-side queue of unsent POST requests while offline
Boot	Transition sequence from auth to main app after token acquisition
Summary	Weekly aggregation (Sunday-based) of sets & weight average

---

12. Future Extensions (Conceptual)

- Multiple user profiles per device (namespaces in storage key).
- Exercise customization UI (persisted to user profile table).
- Cloud sync conflict resolution (merge strategy if multi-device same day updates).
- Analytics events pipeline (privacy compliant) for UX improvement.

End of document.